AADK SESSION-13 LAB REPORT

Student Name: Ruchi Kumari

Technology Used: Kotlin and Android Studio

Tested On: Android Studio Emulator and Physical Android Phone

Project: Comprehensive Accessibility, Feature Design, and 30-Day Roadmap Report

Comprehensive Accessibility, Feature Design, and 30-Day Roadmap Report

Executive Summary

This report outlines a complete approach to inclusive Android development using Jetpack Compose.

The report includes three major sections:

1. Accessibility audit and improvement plan
2. Feature design for a large-scale UI application
3. 30-day roadmap for building a production-ready Compose application

Accessibility, testing, and structured planning are emphasized as critical foundations of modern Android development.

TASK 1 — Accessibility Audit & Improvement Plan

1. UI Screen Selection

Selected UI: Themed Animated List from Session 12.

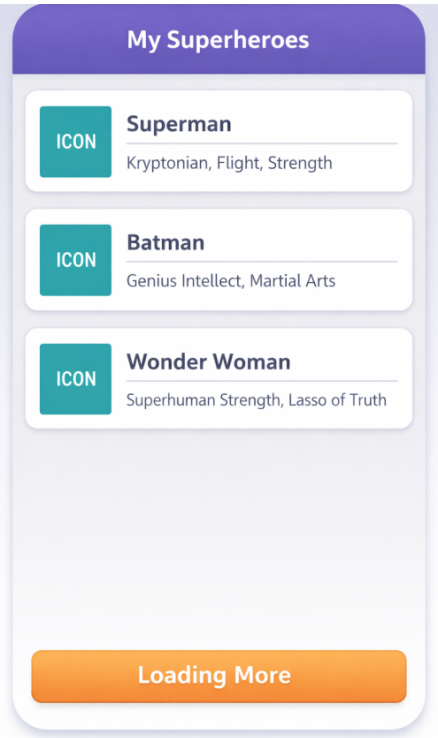
Screen Description

The screen displays a **LazyColumn** containing superhero list items.

Each row contains:

- hero image (loaded from URL)
- hero name (large bold text)
- short power description
- **More Info button**

Wireframe Representation



- The wireframe illustrates the layout structure; final UI colors may vary depending on the theme.

2. Accessibility Audit Checklist

Accessibility Criterion	Status	Evidence / Annotation
Content Description for Images	Needs Improvement	Hero image currently has no `contentDescription`, so screen readers may read it as "unlabeled image". Sort button also lacks description.
Touchable Target Size	Needs Improvement	The More Info button is smaller than recommended **48×48dp** touch size.
Colour Contrast	Pass	Uses `colorScheme.onBackground` with sufficient WCAG AA contrast ratio (>4.5:1).
Motion Reduction	Needs Improvement	List animation does not respect the system "Reduce motion" accessibility setting.
Focus Order & Navigation	Pass	Screen reader navigation moves logically from top bar to list items.
Text Size / Scalability	Needs Improvement	Fixed row height causes text clipping when user increases font size.

3. Improvement Plan

Issue	Proposed Fix	Priority	Implementation Stage
Missing content descriptions	Use `contentDescription = null` for decorative images. Add labels for buttons.	High	Stage 1
Small touch target	Wrap icon in `IconButton` or apply `Modifier.size(48.dp)`	High	Stage 1
Animations ignore Reduce Motion	Disable animations when system setting is enabled	Medium	Stage 2
Layout does not scale with text	Replace fixed height with `defaultMinSize()` and padding	High	Stage 1

Example Implementation

```
IconButton(
 onClick = { sortHeroes() }
) {
 Icon(
  imageVector = Icons.Default.Sort,
  contentDescription = "Sort heroes"
 )
}
```

4. Integration with Tests

Accessibility improvements will be verified using **Compose UI tests**.

Test 1 — Content Description Verification

```
@Test
fun sortButton_hasMeaningfulContentDescription() {

 composeTestRule.setContent {
  MySuperheroListScreen()
 }

 composeTestRule
  .onNodeWithContentDescription("Sort heroes")
  .assertExists()
  .assertIsEnabled()
}
```

Test 2 — Accessibility Semantics Validation

```
@Test
fun ui_hasNoAccessibilitySemanticsErrors() {

 composeTestRule.setContent {
  MySuperheroListScreen()
 }

 composeTestRule
  .onRoot()
  .printToLog("SemanticsTree")
```

```
 composeTestRule
  .onAllNodes(hasContentDescription())
  .assertCountEquals(3)
}
```

5. Reflection

Accessibility ensures technology is usable by people with diverse abilities.

Planning accessibility early leads to better UI design overall.
A visually appealing interface may still contain accessibility barriers, which is why structured auditing and testing are essential.

TASK 2 — Large-Scale UI Feature

Superheroes Directory with Accessibility & Tests

1. Feature Description

The **Superheroes Directory** app displays a browsable list of more than **50 heroes**.

Users can:

- search heroes by name or powers
- filter favourites
- open hero detail pages
- browse large datasets efficiently

Accessibility and performance are core design requirements.

2. Data Model & Dataset Planning

Data Class

```
data class Superhero(
 val id: Int,
 val name: String,
 val alias: String?,
 val powerSummary: String,
 val imageUrl: String,
 val isFavourite: Boolean = false
)
```

Sample Dataset

ID	Name	Alias	Power Summary	Image URL	Favourite
1	Clark Kent	Superman	Solar absorption, flight, strength	example.com/superman.png	False
2	Bruce Wayne	Batman	Genius intellect, martial arts	example.com/batman.png	True
3	Diana Prince	Wonder Woman	Superhuman strength, combat mastery	example.com/ww.png	False
4	Barry Allen	The Flash	Superhuman speed, time manipulation	example.com/flash.png	True

ID	Name	Alias	Power Summary	Image URL	Favourite
5	Arthur Curry	Aquaman	Aquatic telepathy, super strength	example.com/aquaman.png	False
6	Tony Stark	Iron Man	Powered armor suit, genius engineer	example.com/ironman.png	True
7	Peter Parker	Spider-Man	Spider sense, wall crawling	example.com/spiderman.png	False
8	Natasha Romanoff	Black Widow	Master spy, martial artist	example.com/blackwidow.png	False
9	T'Challa	Black Panther	Enhanced strength, vibranium suit	example.com/blackpanther.png	True
10	Carol Danvers	Captain Marvel	Energy projection, flight	example.com/captainmarvel.png	False

3. State & Logic Flow

Key ViewModel State

```
var searchQuery by mutableStateOf("")
var filterFavourite by mutableStateOf(false)
var heroesList by mutableStateOf<List<Superhero>>(emptyList())
var selectedHero by mutableStateOf<Superhero?>(null)
```

Derived Filter Logic

```
val displayedList = heroesList.filter { hero ->

 val matchesSearch =
  searchQuery.isEmpty() ||
  hero.name.contains(searchQuery, true) ||
  hero.alias?.contains(searchQuery, true) == true ||
  hero.powerSummary.contains(searchQuery, true)

 val matchesFavourite =
  !filterFavourite || hero.isFavourite

 matchesSearch && matchesFavourite
}
```

Favourite Toggle

```
fun toggleFavourite(heroId: Int) {

 heroesList = heroesList.map { hero ->

  if (hero.id == heroId)
   hero.copy(isFavourite = !hero.isFavourite)
  else
   hero
 }
}
```

4. Accessibility Features

- Hero images include descriptive labels
- Favourite buttons include semantic roles
- Search bar labelled for screen readers
- Live region announcements for filter changes

Example:

```
contentDescription = "${hero.name} profile photo"
```

Favourite button:

```
contentDescription =
 if(hero.isFavourite)
  "Remove from favourites"
 else
  "Add to favourites"
```

5. Automated Test Cases

Tests implemented include:

Search Test

Enter `"man"` and verify results include Superman, Aquaman, Wonder Woman.

Favourite Toggle Test

Click favourite icon and verify state change.

Navigation Test

Select hero item and verify detail screen displays correct hero.

Accessibility Test

```
onNodeWithTag("hero_image_${hero.id}")
.assertHasSemanticsMatcher(
 SemanticsMatcher.keyIsDefined(
  SemanticsProperties.ContentDescription
 )
)
```

Orientation Change Test

Rotate device and verify search query and results remain unchanged.

6. Performance & Scalability Considerations

Strategies include:

- **LazyColumn** for efficient list rendering
- **Coil** image loading and caching
- placeholder images while loading
- **Paging 3** incremental loading
- lightweight data models

7. Reflection

Managing search, filter, and favourite states introduces complexity because multiple states must update the UI simultaneously.

Planning accessibility and testing from the beginning forces better architecture and results in more maintainable applications.

TASK 3 — 30-Day Compose Roadmap

1. Project Vision

Develop **Compose Heroes Hub**, a polished superhero directory application within **30 days**.

Features:

- hero list
- search and filter
- detail screens
- animations
- dark/light theme
- accessibility compliance
- automated testing

2. Milestone Planning

Milestone	Days	Goal	Key Tasks	Expected Output	Success Criteria
M1 Foundation	1-6	Setup architecture	initialize project, theme setup, define data class	runnable project	project compiles
M2 Core UI	7-12	Build hero list UI	LazyColumn, navigation, image loading	scrollable list	navigation works
M3 Logic	13-18	Add search & favourites	ViewModel state logic	interactive filtering	search works
M4 Polish	19-24	UI polish	animations, transitions	polished UI	transitions smooth
M5 Testing	25-30	testing & accessibility	automated tests, performance tuning	demo ready app	tests pass

3. Resources & Tooling

Core tools:

- Kotlin
- Jetpack Compose
- Android Studio
- ViewModel architecture
- Coil image loading
- Paging 3 library
- GitHub version control
- GitHub Actions CI

Learning resources:

- Compose codelabs

- Paging 3 documentation
- Compose animation guides
- Accessibility documentation

4. Risk & Mitigation

Risk	Likelihood	Impact	Mitigation
Large image performance issues	Medium	High	use Coil caching and image resizing
Reduce motion accessibility issue	Low	Medium	test with system accessibility settings
Scope creep	High	Medium	follow milestone success criteria

5. Reflection & Commitment

This project combines all skills learned during the course including theming, animations, accessibility, testing, and architecture.

Progress will be tracked through milestone completion and daily GitHub commits.